

Hybrid methods for optimizing a linear fractional function over the integer efficient set: what each hybridization is worth, measured

Abstract

Optimizing a linear fractional function Φ over the efficient set of a multi-objective integer program is usually done by repeatedly truncating the *decision* space with Sylva–Crema cuts. Each cut states a disjunction, so it costs p binary variables and $p + 1$ big- M rows, and the sub-problem solved at every iteration is strictly larger than the one before it. We first move the unexplored part of the problem into *criterion* space, as a list of boxes: the truncation becomes a box subtraction, every sub-problem is the original feasible set plus a handful of ordinary linear rows, and the model never grows. On 35 instances this is faster on 34, by $5.05\times$ overall and $9.01\times$ on the heaviest, with the margin widening in p .

We then hybridize that exact search three ways and report what each is worth. Handing it the unfinished work of the published method (exact–exact) *ties* it. Seeding it from a Pareto local search with an archive (exact–metaheuristic) gives $1.25\times$ against a measured ceiling of $1.42\times$. Seeding it from the augmented weighted Tchebychev program — the scalarization the surrounding literature generates efficient points with — *loses* to not seeding at all, at $0.71\times$.

Two further changes sharpen the exact search without altering what it returns. Pruning the open list the moment its best bound cannot beat the incumbent removes a quarter of the sub-problems and no measurable time; stacked on the metaheuristic seed it removes 4% rather than 25%, because the two feed the same bound comparison and are substitutes rather than complements. And not re-deriving a theorem of Ecker and Kouada once per inefficient box removes 6% of the branch & bound nodes at an identical box count. Both are reported in work rather than in seconds, because both effects are smaller than this machine’s noise floor and no number of repeats changes that. A third, warm starting each box from its parent’s basis, removes 11% of the simplex pivots at an identical box count.

The Tchebychev result is the informative one, and it does not say the scalarization is weak. As a generator it is strictly stronger than a weighted sum, reaching unsupported efficient points no positive weighted sum can reach. It is a poor *seed* because it steers in criterion space relative to the ideal point and never looks at Φ . Symmetrically, the same free optimum that saves the box search a third of its work saves the decision-space method *zero* iterations. What a heuristic incumbent is worth is a property of the exact method it is attached to, not of the heuristic; we measure this in both directions. All arithmetic is exact and rational, every answer is proved optimal and cross-checked against independent references, and the negative results and two corrected measurement errors are reported alongside the wins.

Contents

1	Problem	2
2	Where the cost of the decision-space method comes from	3
3	The same truncation in criterion space	3
3.1	How a box is written, and why not with numbers	4
3.2	The split	4

3.3	Bounding, and why it is what makes seeding possible	4
3.4	Stopping when the best bound still open is hopeless	4
4	Two obstacles specific to a fractional objective	5
5	The algorithm	5
5.1	Not paying twice for Ecker & Kouada	6
5.2	Paying the efficiency test less often	6
5.3	Warm-starting a box from its parent	7
6	Computational results: decision space versus criterion space	8
7	First hybridization: exact–exact	9
8	Second hybridization: exact–metaheuristic	9
8.1	The heuristic	9
8.2	Where correctness lives	10
8.3	What it costs, and what it is worth	10
9	The asymmetry: a seed is worth what the exact method can do with it	10
9.1	Two ideas that feed the same mechanism do not add	11
10	Third hybridization: the Tchebychev reference	11
10.1	The program	11
10.2	As a generator it is strictly stronger than a weighted sum	12
10.3	As a seed it loses to not seeding at all	12
10.4	The two generators compared directly	13
11	Delivering a subset of the efficient set	13
12	The complete front, in criterion space	14
12.1	Is the baseline fair? Where the decision-space time actually goes	14
12.2	What the pruning is worth	16
13	From the front to the complete efficient set	16
14	Three measurement lessons, and what they cost	17
15	Summary of what was and was not gained	18
16	Prior work, and what remains to be checked	18
17	Reproducibility	19

1 Problem

Let

$$(P_D) \quad \text{“max”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \quad x \in D = \{x \in \mathbb{Z}_+^n : Ax \leq b\}, \quad (1)$$

with D non-empty and bounded and every denominator $D_k(x) = d_k^\top x + b_k$ strictly positive on D . A point $x \in D$ is *efficient* when no $y \in D$ satisfies $Z(y) \geq Z(x)$ with at least one strict

inequality; $E(P_D)$ denotes the set of efficient points. Linear criteria are the degenerate case $d_k = 0$, $b_k = 1$.

Given the decision maker's criterion $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$, the problem is

$$(P_E) \quad \max \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \quad (2)$$

$E(P_D)$ is a union of faces of D : non-convex, with no explicit description. The point of any method for (P_E) is to reach the global optimum *without enumerating* $E(P_D)$.

An efficient point is *supported* when it maximizes some strictly positive weighted sum $w^\top Z$ over D — equivalently, when its criterion vector lies on the convex hull of the criterion image. The distinction matters throughout Section 10: in an integer program unsupported efficient points are common, and no weighted sum reaches any of them.

2 Where the cost of the decision-space method comes from

One iteration of the published method maximizes Φ over a truncated region D_l , tests the maximizer x_l for efficiency, banks the best Φ on the criterion slice about to be destroyed, and then truncates:

$$\text{delete } \{x : Z(x) \leq Z(\tilde{x})\}. \quad (3)$$

Keeping the complement means keeping the x for which *some* criterion strictly improves on $\tilde{x}$. That is a disjunction, and a disjunction is not a system of linear inequalities. It is modelled with one binary per criterion,

$$e_k(x; \tilde{x}) \geq 1 \cdot y_k + M_k(1 - y_k), \quad k = 1, \dots, p, \quad \sum_{k=1}^p y_k \geq 1, \quad (4)$$

where M_k is a lower bound on e_k over the region, so $y_k = 0$ leaves its row vacuous and $y_k = 1$ forces a strict improvement.

Every cut therefore adds p binaries and $p+1$ rows. On the heaviest instance we ship, eleven cuts leave 33 binaries and 44 rows on top of D , and the late sub-problems cost far more than the early ones. This is the whole of the method's cost profile: the number of iterations is modest, but each one is solved on a strictly larger mixed-integer program than the last.

This observation fixes the cost model used for every comparison in this note:

$$\text{total cost} \approx (\text{number of sub-problems solved}) \times (\text{size of a sub-problem}). \quad (5)$$

The two factors are attacked by different ideas. Section 3 removes the growth of the second. Sections 8 and 10 attack the first, and the measurements there are only interpretable because the second has already been made constant.

3 The same truncation in criterion space

Equation (3) deletes a *box* from $\mathbb{R}^p$. If the unexplored part of the problem is carried as a list of boxes rather than as a region of $\mathbb{Z}^n$, the truncation is a rewrite of that list, and each sub-problem is

$$\max \Phi(x) \quad \text{s.t.} \quad x \in D, \quad \text{the box's rows}, \quad (6)$$

which is D plus at most $2p$ ordinary linear rows. No binary variable appears, and **the model never grows**: a box deep in the search carries a few more rows than a shallow one, and that is all.

3.1 How a box is written, and why not with numbers

The obvious encoding, $l_k \leq Z_k(x) \leq u_k$, fails. In the general case Z_k is a ratio, so its values are rational, and the exact “+1” step that the split below needs would be lost.

Write the box with the linear form of the paper’s Theorem 4 instead:

$$e_k(x; a) = D_k(a) D_k(x) (Z_k(x) - Z_k(a)) = (D_k(a)c_k - N_k(a)d_k)^\top x + (D_k(a)a_k - N_k(a)b_k). \quad (7)$$

Both denominators being positive, e_k carries the sign of $Z_k(x) - Z_k(a)$ while being *linear in x* ; with integer data it is integer-valued on integer points. Hence, exactly and with no minimal step to estimate,

$$Z_k(x) > Z_k(a) \iff e_k(x; a) \geq 1, \quad Z_k(x) \leq Z_k(a) \iff e_k(x; a) \leq 0. \quad (8)$$

Linear and fractional criteria therefore go through the same code, and on a linear criterion e_k collapses to $c_k^\top x - c_k^\top a$.

3.2 The split

Proposition 1. *Let B be a box and $a \in D$. For $k = 1, \dots, p$ put*

$$B_k = B \cap \{x : e_k(x; a) \geq 1\} \cap \bigcap_{j < k} \{x : e_j(x; a) \leq 0\}. \quad (9)$$

Then $B_1, \dots, B_p$ are pairwise disjoint and $\bigcup_k B_k = B \setminus \{x : Z(x) \leq Z(a)\}$.

Proof. Let $x \in B$ with $Z(x) \not\leq Z(a)$. The set of indices at which x strictly beats $Z(a)$ is non-empty; let k be its smallest element. By (8), $e_k(x; a) \geq 1$, and $e_j(x; a) \leq 0$ for every $j < k$, so $x \in B_k$. Conversely a point of B_k has $e_k \geq 1$, hence $Z(x) \not\leq Z(a)$. Disjointness: membership of B_k forces k to be exactly that smallest index, and a point has only one. $\square$

Each of the p children is the parent’s row list plus at most k further rows, all of them ordinary linear inequalities in x .

3.3 Bounding, and why it is what makes seeding possible

The sub-problem (6) maximizes Φ over a *superset* of the efficient points whose criterion vector lies in the box, so its value bounds all of them. A box whose bound does not beat the incumbent is therefore discarded *whole* and never split. Children inherit their parent’s value as a bound, which makes the open list a certified upper bound on everything still unfound; the method is *anytime* in the same sense, and for the same reason, as the decision-space one.

This is the single structural fact that Sections 8 and 10 rest on. A better incumbent does not merely stop the search sooner; it deletes subtrees. The decision-space method has no such mechanism, which is measured in Section 9 and is the reason the two methods respond so differently to the very same incumbent.

3.4 Stopping when the best bound still open is hopeless

The open list is kept in a heap ordered by inherited bound *descending*, and only the root — and the boxes a hybrid hands over — carry no bound; those rank first. So by the time a bounded box reaches the top, every box still open is bounded too, and if this one cannot beat the incumbent, neither can any of them. The search is finished, and the remaining list is busywork.

Measured before the change was made, on the 18 instances used throughout Sections 8–10: 198 of 733 sub-problems (27%) were solved despite carrying an inherited bound the incumbent already beat — and on *every single instance* that count equalled the number of boxes still open

when the condition first fired. The waste is exactly a tail, never scattered, which is what the heap order predicts and which is why a single early exit is the whole of the fix rather than a per-box filter. Stopping there takes 733 boxes to 549, **25% fewer**, with the same answers, all still proved optimal.

It is not a speed-up, and we do not report it as one. The total is 7.44s against 7.27s at per-instance minima over five repeats, with per-instance ratios scattered from $0.77\times$ to $1.36\times$. The change can only *remove* work, so every ratio below $1.00\times$ is machine noise — and the noise is larger than the effect.

That is a confirmation rather than a disappointment, and it sharpens the cost model of (5). The tail boxes are the *cheapest* ones: each is killed by the root relaxation the instant it is handed the cutoff, so a quarter of the sub-problems were genuinely wasted and worth almost nothing. Where the count of sub-problems and the cost of the search come apart, it is the count that is the easier number to improve and the weaker one to quote.

4 Two obstacles specific to a fractional objective

Criterion-space search is a known family for optimizing a *linear* function over the efficient set. It does not transfer to (P_E) by inspection, for two reasons.

Remark 1 (Φ is a function of x , not of $Z(x)$). *Two feasible points with the same criterion vector may have different values of Φ . Enumerating non-dominated vectors and evaluating Φ on a representative of each is therefore wrong. Inside every box a genuine fractional integer program must be solved, and before cutting on an efficient centre $\tilde{x}$ the sub-problem $Q(\tilde{x}) = \max\{\Phi(x) : x \in D, Z(x) = Z(\tilde{x})\}$ must be solved first, since the whole slice $\{Z = Z(\tilde{x})\}$ is efficient and is about to be removed.*

Remark 2 (Box boundaries must be integer-valued). *As argued above, numeric bounds on Z_k are rational in the fractional case. Writing the boundaries through e_k restores an exact integral threshold and, as a by-product, makes fractional criteria work with no special case.*

5 The algorithm

Termination: every pass removes at least the criterion vector $Z(a)$, and the non-dominated set of a bounded integer program is finite.

Line 2 is the whole of the hybridization interface. Everything in Sections 8 and 10 is a different way of filling it, and the correctness requirement is stated once, in Proposition 2 below: the seed must be *efficient*, not merely good.

Proposition 2 (Soundness of an arbitrary seed). *If the seed x_0 in line 2 is efficient, Algorithm 1 returns the same $(x_{\text{opt}}, \Phi_{\text{opt}})$ as the unseeded run, and Φ_{opt} is the global optimum of (P_E) . If x_0 is merely feasible, the algorithm may return a strictly suboptimal value.*

Proof. Φ_{opt} is used only as a cutoff, and a box is discarded when its bound fails to *beat* it. If $x_0 \in E(P_D)$ then $\Phi(x_0) \leq \max_{x \in E(P_D)} \Phi(x)$, so no box containing an optimal criterion vector is ever discarded, and the incumbent is a feasible value of (P_E) throughout; the two runs therefore agree. If $x_0 \notin E(P_D)$ then $\Phi(x_0)$ may exceed the optimum of (P_E) — a dominated point can score arbitrarily well on Φ — and the box containing the true optimum is discarded by the very first bound comparison. $\square$

This is why every seed in this note is *verified* before it is used, and why that verification, not the heuristic, turns out to be the cost of Section 8.

Algorithm 1 Criterion-space search for (P_E) , optionally seeded

```
1:  $L \leftarrow \{\mathbb{R}^p\}$ 
2:  $(x_{\text{opt}}, \Phi_{\text{opt}}) \leftarrow$  a verified efficient seed and its value, or  $(\emptyset, -\infty)$ 
3: while  $L \neq \emptyset$  do
4:   remove from  $L$  a box  $B$  of largest inherited bound
5:   solve (6) on  $B$  with cutoff  $\Phi_{\text{opt}}$ 
6:   if infeasible, or its value  $\leq \Phi_{\text{opt}}$  then
7:     continue  $\triangleright B$  is discarded whole
8:   end if
9:   let  $x_B$  be its maximizer
10:  if  $x_B$  is efficient then
11:     $\Phi_{\text{opt}} \leftarrow \Phi(x_B)$ ;  $x_{\text{opt}} \leftarrow x_B$ ;  $a \leftarrow x_B$ 
     $\triangleright x_B$  already maximizes  $\Phi$  on its whole slice, so no  $Q$  is owed
12:  else
13:     $a \leftarrow$  an efficient point dominating  $x_B$ 
14:    solve  $Q(a)$  and update  $(\Phi_{\text{opt}}, x_{\text{opt}})$ 
15:  end if
16:  add to  $L$  the  $p$  children of  $B$  given by (9)
17: end while
18: return  $(x_{\text{opt}}, \Phi_{\text{opt}})$ , proved optimal
```

5.1 Not paying twice for Ecker & Kouada

When a box's maximiser x_B fails the efficiency test, the algorithm needs an efficient point to cut on in its place, and the obvious way to get one is to walk the dominance chain from the test's witness until a test returns $\theta = 0$. With *linear* criteria that walk is over before it starts, and the program that discovers this can be skipped.

Proposition 3. *With linear criteria, every optimal solution of the efficiency test applied to x^* is efficient.*

Proof. With $d_k = 0$ the test maximizes $\sum_k e_k(y; x^*) = \sum_k (Z_k(y) - Z_k(x^*))$, a strictly positive weighted sum of Z up to an additive constant, over $R = \{y \in D : Z(y) \geq Z(x^*)\}$. Let y^* be a maximizer and suppose $z \in D$ dominated it. Then $Z(z) \geq Z(y^*) \geq Z(x^*)$, so $z \in R$; and $\sum_k Z_k(z) > \sum_k Z_k(y^*)$ because at least one component is strictly larger. That contradicts optimality, so no maximizer is dominated — ties included, since the argument never assumes y^* unique. $\square$

The proposition is due to Ecker and Kouada; what is measured here is the cost of not using it. With *fractional* criteria it is false, and the proof says where: the k -th term of the objective carries a factor $D_k(y)$ that varies from point to point, the objective stops being monotone in Z , and a dominating point can score lower than a dominated one. There the walk is paid.

Checked before being relied on: 349 witnesses of failed tests across 60 linear-criteria instances, none of them dominated.

The programs removed are *cheaper than average* — 8.5% of the count against 6.0% of the nodes — because proving $\theta = 0$ on a point already known to be efficient is easier than searching for a dominator. The same shortcut applies to the decision-space method, which uses the same test, so the comparison of Section 6 is unaffected by it.

5.2 Paying the efficiency test less often

Splitting the work by operation puts the cost somewhere the wall-clock profile did not: the efficiency test is **52% of all simplex pivots on 146 calls**, 58 pivots each against 11.8 for a

Table 1: Skipping the redundant test, over the 18 instances. The search is *identical* — same centres, same boxes, same answers — so only the price of reaching each centre moves.

	before	after	
integer programs solved	1466	1341	−8.5%
branch & bound nodes	8105	7618	−6.0%
boxes solved, optima	identical		

box sub-problem. The useful question is therefore not how to make the test cheaper but how to pay it less often.

When a box’s maximiser x_B is inefficient, the test is what produces the efficient point to cut on. But the search already holds the points it has *proved* efficient, and asking whether one of them dominates x_B is arithmetic rather than an integer program. One does on 21 of 146 tests (14%), and on 17% of the inefficient cases.

Table 2: Cutting on a remembered efficient point when one dominates the box’s maximiser.

	before	after	
simplex pivots	16169	14602	−9.7%
boxes solved	549	534	−15
optima	identical, all proved		

The box count *falling* was not the prediction. Among the remembered points dominating x_B the highest is taken, and (9) removes more of the box the higher its centre, so skipping the test and cutting better compound instead of trading off.

The invariant this rests on. Every point the search remembers is efficient: it remembers the centres it cut on and, beside each, the maximiser of Q , which shares its centre’s criterion vector and is therefore efficient whenever the centre is. One inefficient member would let the search cut on a dominated point and remove the true optimum, so this is pinned by a test rather than by an argument. Unlike the two changes above, this one deliberately *changes* which boxes are produced, so it is checked against the answer and the proof rather than against a box count.

5.3 Warm-starting a box from its parent

A box’s region is its parent’s plus a handful of rows, yet each box solved its root relaxation from scratch. A simplex phase I is needed whenever a row is not covered by a slack column, which every $e_k \geq 1$ row of (9) forces. Measured before anything was built: **4992 of 18193 simplex pivots, 27%, were phase I inside cold roots**, and 788 of 824 roots paid one — 6.3 pivots each against 2.3 for phase II.

The parent’s basis stays a basis of the child once the new rows’ slacks join it, so only those rows can be primal infeasible and a dual restoration repairs them. That is the argument branch & bound already uses for its own nodes, applied one level up; all it needs is the general form of the single-variable row that branching adds.

The ceiling was never 27%. Only a box with a parent can inherit one, and that is 63% of the roots. The rest are the efficiency tests and the Q sub-problems, which extend D rather than another box — and caching a basis for D would buy nothing, because its rows are all $\leq$ with non-negative right-hand sides, so its crash basis is free and phase I is skipped outright. The reachable ceiling was about 17%, of which 11.1% survives the restoration’s own 1583 pivots.

Table 3: Warm-starting each box from its parent, over the 18 instances.

	before	after	
simplex pivots	18193	16169	−11.1%
cold root relaxations	824	307	−63%
boxes, programs, optima	identical		

The gain grows with p (28% at $p = 7$ against 1–7% at $p = 3$), which is what more rows per box predicts, and two very small instances lose a little where the restoration costs more than the phase I it replaces.

517 warm starts, 0 stalls. The fallback to a cold solve never fired. It is kept regardless, because a restoration that gives up must cost time and never correctness — which is also why the box counts above are identical rather than merely equal in the answers they produce.

6 Computational results: decision space versus criterion space

All figures come from an exact rational implementation with no floating-point arithmetic anywhere, so every comparison below is between two exactly solved problems. Both methods were run on the same instances in the same process.

Table 4: Decision space versus criterion space, $p = 3$.

instance	decision space	criterion space	ratio
35 instances, total	40.65 s	8.06 s	5.05×
heaviest $n = 10$	23.47 s	2.61 s	9.01×
$n = 8$, hardest seed	2.97 s	0.45 s	6.56×
$n = 10$, easiest seed	0.36 s	0.41 s	0.86×

The box search is faster on 34 of the 35, and the single regression is the instance the decision-space method settles in three iterations, where paying 13 boxes is a net loss. Every answer agrees with the decision-space method and with an independent scan, and both are proved optimal everywhere.

Table 5: The margin against the number of criteria, $n = 6$, four instances each. All twenty agreed and were proved optimal on both sides.

p	decision space	criterion space	ratio
2	0.61 s	0.35 s	1.74×
3	1.02 s	0.34 s	2.99×
4	4.21 s	0.61 s	6.92×
5	112.77 s	1.09 s	103×
6	26.15 s	1.08 s	24.3×

Two forces grow with p and pull against each other: the split produces p children, so the list branches wider; but the non-dominated set grows too, so the decision-space method needs more cuts, and each cut costs it p binaries and $p+1$ rows. The measurement settles which dominates, and it settles it against the prediction we made before running it. Branching multiplies a *count* of cheap, fixed-size models; the binaries multiply the *cost of one model*, and they accumulate.

The worst case observed for the published method was $p = 5$ on six variables: 110.95 s against 0.64 s, a factor of 173.

On the magnitudes. The direction is solid: the per- p median ratios are 1.35, 3.2, 2.1, 12.6, 15.8, and every instance agreed and was proved optimal. The totals are not stable estimates — four instances per p is few, and 110.95 of the 112.77 seconds at $p = 5$ come from a single instance. The tables report what was measured, not expected values.

7 First hybridization: exact–exact

Since the published method closes in one iteration whenever the maximizer of Φ over D happens to be efficient, a hybrid suggests itself: run its loop, and if it has not closed after s iterations, hand the unfinished work to the box search. The handover is not a restart. Replaying each cut as a box subtraction (Proposition 1 applied to every box of the current list) reproduces the region the first phase reached, and the incumbent, being attained at a known efficient point, is a valid seed by Proposition 2.

Measured on 24 instances against the better of the two pure methods on each:

	total	within 15% of the better
$s = 3$	13.14 s	4 of 24
$s = 1$	8.47 s	23 of 24
pure box search	8.11 s	—
the published method	268.61 s	—

We report this as a negative result. At $s = 1$ the hybrid *ties* the pure box search — 8.47 s against 8.11 s here, and 1.47 s against 1.54 s on a family built so that Φ is easy; two differences of about 4% pointing opposite ways. The criterion-space search is what matters; this hybrid earns its place only as insurance for the one-iteration case, not as a third method that beats both. The first default we chose, $s = 3$, was measurably wrong and is reported here because it was: 13.14 s against 8.47 s.

8 Second hybridization: exact–metaheuristic

By Proposition 2 the box search will accept any efficient point as a seed, and by Section 3.3 a good one deletes subtrees. That makes a metaheuristic the natural first phase — it is cheap and it is guided by Φ , which is exactly the quantity the cutoff compares against.

8.1 The heuristic

A Pareto local search maintains an archive of mutually non-dominated points. From several random maximal starting points it walks the neighbourhood of each archive member, admitting any neighbour the archive does not dominate. The archive *is* the search: restarting from fresh random points between walks throws it away, and four times the effort did not compensate for that (obtaining 0.360, 0.280 and 0.458 of the optimum on three instances where the archive-driven version obtained the optimum).

The neighbourhood needs *swap* moves (−1 on one coordinate, +1 on another), not only unit steps. From a maximal point a unit step up is infeasible, and a unit step down lowers every criterion with non-negative coefficients, so the archive refuses it and the walk dies immediately: three archive points and an incumbent at 13% of the optimum. With swaps the archive holds 28–69 points and the incumbent reaches 60–100%.

8.2 Where correctness lives

The archive is filtered by dominance *among the points it has seen*, which is not the exact efficiency test. Proposition 2 makes that gap fatal if ignored, so nothing crosses unverified: the best archive member is put through the exact test, and if it fails, it is walked to an efficient point dominating it and that point’s value is used instead. The heuristic decides *where to look*; it never decides what is true.

The gap is not hypothetical. Over a wide sweep, one archive member in roughly 2000 turned out not to be efficient — reproducibly, a concrete case being a four-variable instance whose archive retains (2, 1, 1, 1). Rare is not never.

8.3 What it costs, and what it is worth

The cost is *not* the walk, which takes 0.01–0.08 s and is barely sensitive to its budget. We assumed the opposite and optimized the walk first; the timings then showed the total *falling as the walk’s budget rose*, because a poor archive yields candidates needing longer repair chains, each one an integer program. The cost is the verification — the price of Proposition 2. Examining the single best archive member gave exactly the same incumbent as examining four on every instance tried, at a quarter to a half of the cost.

Table 6: Exact–metaheuristic hybrid, 18 instances. Same optimum everywhere, all proved optimal.

instance	exact only	metaheuristic + exact	ratio
total, 18 instances	9.50 s	7.60 s	1.25×
hardest $n = 10$	49 boxes, 3.02 s	34 boxes, 1.94 s	1.56×
$n = 8$, seed 0	16 boxes, 0.53 s	10 boxes, 0.34 s	1.53×
$p = 7$, seed 1	197 boxes, 1.36 s	120 boxes, 0.95 s	1.43×
$p = 5$, seed 1	6 boxes, 0.02 s	6 boxes, 0.03 s	0.56×

It wins where the search is expensive and loses where there was nothing to win, which is what a nearly fixed first-phase cost predicts. The *ceiling*, measured by handing the search the true optimum for free, is 1.42×; at 1.25× the heuristic realizes 88% of it.

9 The asymmetry: a seed is worth what the exact method can do with it

The ceiling experiment can be run on the published method too, and this is the comparison we consider the most transferable result in this note.

Table 7: The *same* free optimum, handed to each exact method at iteration 1.

exact method	effect of a free optimal incumbent	mechanism
criterion-space box search	1.42×, up to 52% fewer boxes	boxes discarded whole
decision-space cut method	zero iterations saved	none

The reason is structural. What the decision-space method cuts at iteration l is decided by the efficiency test applied to x_l , through (3); Φ_{opt} enters the sub-problem only as a cutoff inside a branch-and-bound that would have found the same x_l anyway. A box, by contrast, is compared against the incumbent *before* it is ever split, and failing that comparison deletes it together with every descendant it would have had.

So “hybridize an exact method with a metaheuristic” is not one question with one answer. The heuristic is identical in both rows above; what differs is whether the exact method has a mechanism through which a lower bound can remove work. Before building such a hybrid, the mechanism is what should be checked.

9.1 Two ideas that feed the same mechanism do not add

The same reasoning has a consequence that is easy to miss when improvements are collected one at a time. The early exit of Section 3.4 and the metaheuristic seed of Section 8 look independent — one prunes at the top of the list, the other starts the incumbent higher — but they act through the *same* mechanism, the bound comparison of Section 3.3. So they should not be expected to compose, and measured, they do not:

Table 8: Sub-problems solved over the 18 instances, with each idea alone and with both.

	boxes solved	reduction
neither	733	—
early exit only	549	25%
metaheuristic seed only	562	23%
both	539	26%

Either alone removes about a quarter of the sub-problems; together they remove about a quarter. On top of the seed, the exit is worth 4% rather than 25%, because a good incumbent arrives early enough that the hopeless tail barely forms — there is nothing left to skip. They are **substitutes, not complements**.

We record this because the arithmetic of stacked improvements is where plausible reasoning goes wrong most cheaply. Two ideas add only when they feed different mechanisms; two that feed the same one are one idea, measured twice.

10 Third hybridization: the Tchebychev reference

The surrounding literature generates efficient points with the augmented weighted Tchebychev program, so a fair evaluation of the two hybrids above has to be made against it and not only against a weighted sum. We therefore implement it and measure it, rather than cite it.

10.1 The program

With z^* the ideal point, $z_k^* = \max\{Z_k(x) : x \in D\}$, weights $w > 0$ and an augmentation $\rho > 0$:

$$\min_{x \in D} \max_k w_k (z_k^* - Z_k(x)) + \rho \sum_{k=1}^p (z_k^* - Z_k(x)), \quad (10)$$

which linearizes with a single continuous variable λ subject to $\lambda \geq w_k (z_k^* - Z_k(x))$ for each k . Each weight vector therefore costs one integer program on a model with $n + 1$ variables and p extra rows, plus p programs paid once for z^* .

Proposition 4. *For any $w > 0$ and any $\rho > 0$, every optimal solution of (10) is efficient.*

Proof. Let x^* be optimal and suppose some $y \in D$ dominated it, so $Z(y) \geq Z(x^*)$ with at least one strict inequality. Then $z_k^* - Z_k(y) \leq z_k^* - Z_k(x^*)$ for every k , so the max term does not increase; and since $\rho > 0$ and at least one inequality is strict, the sum term *strictly* decreases. The objective at y would be strictly smaller than at x^* , contradicting optimality. $\square$

The augmentation is exactly what makes the second half of that argument available: with $\rho = 0$ the objective at y need only be no larger, and a merely *weakly* efficient point can minimize the program. Our implementation therefore refuses $\rho \leq 0$ and refuses a zero weight rather than returning a point whose status is in doubt. It also refuses fractional criteria, for which $z_k^* - Z_k(x)$ is not linear and the rows above are not constraints of an integer linear program; the references assume linear criteria as well.

10.2 As a generator it is strictly stronger than a weighted sum

A weighted sum maximizes only at supported efficient points. Varying w in (10) reaches unsupported ones too, and that is what the literature uses it for.

Table 9: Reach from the same grid of 27 weight vectors, 20 random instances.

	efficient points reached	of which unsupported
weighted sum	66/175	0/54
augmented Tchebychev	112/175	29/54

All 540 optima returned were efficient, as Proposition 4 requires. A wider sweep over 40 instances agrees: 216/369 against 135/369, 56 unsupported against 0, and 1080 optima with no inefficient one. On the reference instance of the published paper, three of the seven efficient points are unsupported; the Tchebychev program reaches all three and no strictly positive weighted sum reaches any.

10.3 As a seed it loses to not seeding at all

Table 10: Seeding the box search, 18 instances. “Exactly optimal” counts the instances where the seed already attains Φ^* .

seed	total	seed cost	exactly optimal
none (pure box search)	9.64 s	—	—
Pareto local search	7.97 s (1.21×)	0.57 s	12 of 18
augmented Tchebychev	13.50 s (0.71×)	4.25 s	5 of 18

Both halves of the loss are structural rather than artefacts of the implementation.

Cost. It is $p + 1$ integer programs on an enlarged model. It is *not* the reference point: computing z^* took 0.01–0.13 s of the seed against 0.13–0.85 s for the scalarizations themselves, so replacing z^* by a cheaper reference would not change the verdict.

Quality. This is the real finding. Program (10) steers by criterion-space geometry relative to z^* and **never looks at Φ** . The local search of Section 8 is guided by Φ at every move. A good spread of efficient points is not a good Φ , and by Remark 1 it cannot be made into one by post-processing: Φ is not a function of $Z(x)$, so two points the generator regards as interchangeable may differ arbitrarily in the only quantity the search cares about.

We therefore keep the Tchebychev program as the reference the evaluation needs, and do not wire it into the default hybrid.

Table 11: As generators of a subset of $E(P_D)$, 20 instances at $n = 4$.

	coverage of $E(P_D)$	time	certification
augmented Tchebychev	64%	2.1 s	every point efficient by construction
Pareto archive	99%	0.04 s	≈ 1 member in 2000 not efficient

10.4 The two generators compared directly

Cheap, near-complete and uncertified against expensive, partial and certified. That is the trade the exact–metaheuristic hybrid closes: it takes the cheap, near-complete generator and buys the missing certificate with a single exact test, which is why the verification and not the walk is its dominant cost.

This also delimits the expected result of the thesis — “a subset of efficient solutions generated by hybrid algorithms”. Both columns produce such a subset, and they are not interchangeable: one is a certified partial sample of the front, the other a near-complete sample whose membership is a conjecture until tested.

11 Delivering a subset of the efficient set

Everything above answers one question — which efficient point maximises Φ — and proves the answer. The work this note implements states a second one as its expected result: *a subset of efficient solutions, generated by hybrid algorithms*. That subset is what a decision maker looks at, and the machinery could already produce it without ever exporting it.

Two sources exist, and a third was measured and rejected.

Table 12: What each source contributes to a delivered subset, over 25 instances small enough to enumerate ($|E(P_D)| = 207$).

source	coverage of $E(P_D)$	cost
the exact search’s own trail	51%	free
the Pareto archive, verified	98–100%	0.07 s + certification
augmented Tchebychev	39%	0.60 s

The exact search must prove a point efficient before it can cut on it, so its trail is certified and costs nothing beyond the search itself. The archive of Section 8 is what makes the subset nearly complete. And the Tchebychev generator of Section 10, certified by construction and the stronger generator in the reach experiment there, contributes *not one additional point* here — every point it reached the archive had already found. Its exclusion from the default is measured, not assumed.

Certification is the cost. The archive filters by dominance among the points it has *seen*, which is not the exact efficiency test; about one member in 2000 is not efficient. Certifying the whole archive is one integer program per member — 0.25 s against a 0.03 s walk at $n = 4$, 0.62 s against 0.11 s at $n = 5$ — and it is the same finding as Section 8’s, one level up: the walk is cheap and the proof is not.

What is not claimed. This is a subset and completeness is never asserted. The coverage figures are measured against exhaustive enumeration on instances small enough to enumerate; where $E(P_D)$ cannot be enumerated, what fraction was found is unknowable, and the implementation says so rather than reporting a percentage it cannot support. The exact search’s trail

is also a *biased* sample — it is precisely the points the search had to cut on, so it concentrates where Φ is large. For a decision maker ranking by Φ that bias points the useful way, but it is a bias and is reported as one.

12 The complete front, in criterion space

Section 11 delivers a subset and refuses to claim completeness. That refusal is honest but it is also a limitation, and the same substitution this note is built on removes it.

Complete enumeration of the non-dominated set was already available, but in *decision* space: truncate D with Sylva–Crema cuts, pick a point, repair it, cut again. Every cut costs p binaries and $p + 1$ big- M rows, so the sub-problems grow — exactly the cost profile Section 3 replaces. Keeping the unexplored region as a list of boxes instead gives the same enumeration with no binary anywhere and no model growth.

Table 13: Enumerating the whole front, six instances per row.

instance	front	criterion space	decision space	ratio
$n = 4$	9	0.12 s	1.37 s	11.4×
$n = 5$	30	0.63 s	129.21 s	204 ×
$n = 5$	9	0.29 s	0.81 s	2.8×
$n = 6$	6	0.08 s	0.63 s	8.0×

The ratio tracks the size of the front, not n . Every vector is a cut, and in decision space every cut adds p binaries and $p + 1$ big- M rows, so a 30-vector front leaves 90 binaries on the last sub-problem while the box list never grows. The instance with the largest front is the one with the 204× gap; the two smallest fronts give 2.8× and 8.0×.

That is the same mechanism as Section 6, where the margin tracked the number of cuts rather than n — measured here independently, on a different task. *Four instances, and the spread is wide*: the mechanism explains it, but these are four runs and not a distribution.

12.1 Is the baseline fair? Where the decision-space time actually goes

A 204× margin invites the suspicion that the loser was handicapped, so we profiled it rather than quoting it. On the $n = 5$ instance, of 137s spent producing 30 vectors, 137.21 s — **100%** — **is inside the probe**, over 31 calls and 14983 branch & bound nodes; the repair takes 0.09s and adding the cuts 0.02s. The model grows from 5 columns and 7 rows to 95 and 127, which is exactly p binaries and $p + 1$ rows per cut. The cost is therefore structural — it is the growth of the sub-problems, which is the thing Section 3 removes — and not an artefact of some inefficiency on the side of the comparison.

One candidate handicap remained. The probe only needs *some* feasible point of the truncated region, yet it maximises a linear direction; stopping at the first integer point found looks strictly cheaper. Measured, it is the opposite:

Table 14: Probing by maximising a direction against probing with a zero objective.

	cuts	B&B nodes	time
$n = 4$, maximise	9	658	1.35 s
$n = 4$, zero	9	1397	5.33 s
$n = 5$, maximise	9	372	0.79 s
$n = 5$, zero	9	2105	10.15 s

The **cut counts are identical**: the direction changes neither how many vectors there are nor the order they come out in. What explodes is the branch & bound inside each probe, by a factor of two to six, because a zero objective gives it nothing to prune with — every node’s relaxation is worth 0, so no bound can discard a subtree until an integer point has been stumbled upon. The direction is load-bearing rather than decorative, the criterion-space method uses the same one, and the $204\times$ stands.

We report this because it is a plausible-looking optimisation that makes things four to thirteen times worse, and because it was checked while suspecting the *comparison* was unfair — it was not.

Proposition 5. *The enumeration terminates, and records every non-dominated vector exactly once.*

Proof. Termination. The probe returns a point x of D inside the box, and the efficient point a it is repaired to dominates it, so $Z(x) \leq Z(a)$ and the split around a removes at least $Z(x)$ from that box. The unexplored region therefore shrinks on every pass, and the non-dominated set of a bounded integer program is finite.

Completeness. A non-dominated vector v leaves the region only through a split around some centre a with $v \leq Z(a)$. If $v \neq Z(a)$, that says a dominates v , which no non-dominated vector admits. So v leaves only at the moment it is recorded; and the loop ends only when the region is empty, so every v is recorded, exactly once since the centres are de-duplicated by criterion vector. $\square$

Completeness is a flag, not a default. It is set only when the box list actually emptied. A run stopped by a box budget or a time budget returns what it has with the flag *false* — a front that cannot say it is whole is a subset with extra steps, and saying so is the difference between the two sections.

The front is not the set of efficient points. One non-dominated vector can be attained by several efficient points, and the enumeration keeps *one per vector*: the front is complete while the set of points it carries need not be. On random instances the distinction never appeared — over 32 of them, 476 efficient points sat on 476 distinct vectors — but that is a property of the sample, since generic criterion coefficients make ties improbable. Constructed deliberately, with a criterion that ignores a variable, $|E(P_D)| = 6$ sits on 2 vectors and the enumeration returns 2 points of the 6.

That instance also shows what Q is for. Its slice $Z = (1, 2)$ holds three efficient points with $\Phi = 3/4, 6/5$ and $3/2$; the repair lands on the first, and Q moves it to the third. So the largest value on the front is still the optimum of (P_E) — and without Q the answer would have been *half* of it, with the front itself still correct and the output looking entirely reasonable.

Vectors are not solutions. By Remark 1, Φ is a function of x and not of $Z(x)$, so one non-dominated vector can carry several efficient points with different Φ : enumerating the front does not by itself answer (P_E) . Each vector is therefore paired with the maximiser of Φ *on its slice*, at one extra integer program per vector — the pairing a decision maker wants, and the one a plain vector enumeration cannot provide.

That pairing has a consequence worth stating, because it closes the circle of this note: **the largest value on the front is the optimum of (P_E)** . The front enumeration is therefore a second, independent exact method for the problem, and the test suite uses it as a cross-check on the first.

12.2 What the pruning is worth

Two exact routes to (P_E) now exist: *prune* — stop as soon as no open box can beat the incumbent — or *enumerate* the whole front and take the best value on it. They share the box machinery but not the reason they stop, so the comparison measures the pruning itself.

Table 15: Solving (P_E) by pruning against solving it by enumeration, eight instances at $n = 6, 7$. Same optimum on every one.

	boxes	time
prune (Algorithm 1)	102	1.52 s
enumerate (Section 12)	215	2.96 s

1.94 $\times$, and half the sub-problems. That is *less* than one might expect: walking the entire front costs only twice what finding its single best point costs, which says the cutoff prunes a smaller share of the work than the ceiling experiments of Section 9 might suggest — those measure what a better *incumbent* saves, not what having one at all does.

13 From the front to the complete efficient set

Section 12 proves its vector list complete and says, in the same breath, that the *point* list is not: one vector is kept per slice. That was the last honest gap in the chain, and closing it turns out to cost almost nothing, for a reason worth stating on its own.

Proposition 6. *If v is non-dominated, then every $x \in D$ with $Z(x) = v$ is efficient.*

Proof. Suppose some $y \in D$ dominated such an x . Then $Z(y) \geq Z(x) = v$ with at least one strict inequality, which says v is dominated — against the hypothesis. So no y dominates x . $\square$

Two consequences, and together they are the whole method. First,

$$E(P_D) = \bigcup_{v \text{ non-dominated}} \{x \in D : Z(x) = v\},$$

so the complete front plus its slices *is* the complete efficient set. Second, **not one efficiency test is paid on any slice point**: they are efficient by Proposition 6, not by examination. The expensive operation of this whole note — the efficiency test of Section 5.1 — is simply not invoked.

A slice is D plus p equalities, and nothing more. Fixing $Z(x) = v$ is linear in x *even for fractional criteria*: since $D_k(x) = d_k^\top x + b_k > 0$ on D , the equation $(c_k^\top x + a_k)/(d_k^\top x + b_k) = v_k$ clears to

$$(c_k - v_k d_k)^\top x = v_k b_k - a_k,$$

with no sign condition to check and no case split. So a slice carries no binary, no big- M , and nothing that grows from one slice to the next — the same property the box search has, arrived at a second time. The equalities are in fact the *tightest* rows in the system, which is what makes the depth-first scan over them cheap.

What it recovers, and what it costs. The family below fixes five variables that enter Z and adds f further variables that enter only their own bound $0 \leq x_j \leq 4$. Those are invisible to Z , so every slice is a grid of 5^f points and the front keeps one of them.

At $f = 3$ the front carries 0.8% of the efficient set — it is complete and it is missing 99.2% of the points, which is exactly the distinction Section 12 insisted on — and recovering all 1625 of

Table 16: The complete efficient set against the front alone. “Overhead” is the extra time over enumerating the front, bounding programs included.

f	front	$ E(P_D) $	ratio	front	slices	overhead
0	7	7	$1\times$	0.110 s	0.016 s	24%
0	13	13	$1\times$	0.247 s	0.018 s	10%
1	7	35	$5\times$	0.114 s	0.019 s	27%
1	13	65	$5\times$	0.240 s	0.024 s	13%
2	7	175	$25\times$	0.117 s	0.036 s	42%
2	13	325	$25\times$	0.246 s	0.050 s	24%
3	7	875	$125\times$	0.128 s	0.103 s	90%
3	13	1625	$125\times$	0.259 s	0.167 s	68%

them **never doubles the cost**. The scan is output-sensitive: 0.167 s for 1625 points is about 0.1 ms each, against the 7.68 ms an efficiency test costs, and there are no tests to pay.

The honest reading, which is conditional. At $f = 0$ the front *already was* the efficient set, and the 10–24% buys only the proof of that. On generic random instances this is the usual case: over 32 of them, 476 efficient points sat on 476 distinct vectors. So the value of this section is not a speed-up and is not uniform. It is that the method now *knows* which case it is in, instead of returning a point list whose completeness depended on an assumption about the coefficients. Where ties do occur — criteria that ignore a variable, repeated structure, any instance that is not generic — the front alone silently returns a fraction of the answer, and the fraction can be under one percent.

What is verified. The complete set is checked against exhaustive enumeration on two families, one generic and one built to force ties (24 instances, exact match, 21 non-singleton slices). Proposition 6 is checked directly rather than assumed: 74 points on non-dominated slices, every one efficient. The front’s own representative is checked to lie inside its slice. And, as everywhere else in this note, an exhausted budget returns `complete = False` rather than a set that claims what it did not prove.

14 Three measurement lessons, and what they cost

We report these because the direction of a result is worth less than the knowledge of how it was checked.

An unsound quality metric. Seed quality was first scored by the ratio $\Phi(\text{seed})/\Phi^*$. That ratio *inverts when Φ^* is negative*, which it is on several instances in these families: a strictly worse seed can score above 1. All seed-quality figures in this note use the non-negative gap $(\Phi^* - \Phi(\text{seed}))/|\Phi^*|$, which is well defined whatever the sign and is zero exactly when the seed is optimal. No published number changed direction under the correction, but the metric was wrong and one table read 1.286 for a seed that was in fact 29% below the optimum.

Seed quality is not seed value. On a looser instance family, the Pareto seed is *exactly optimal on 15 of 24 instances* and still buys only $1.05\times$, because that search is not bound-limited: there are few boxes whose bound the incumbent can defeat. Quality is a property of the heuristic; value is a property of the pair. What is stable across all families we tried is the *ordering* — Tchebychev loses to no seed at all ($0.73\times$, $0.91\times$), the Pareto seed ties or wins —

not the size of any win. Single speed-up figures quoted without their instance family should be read with this in mind, including ours.

Below the noise floor, count work and not seconds. The change of Section 5.1 removes 125 integer programs, and its wall-clock A/B reads $0.96\times$ — *slower* — faster on 9 of 18 instances. That cannot be causal: removing work cannot cost time. The machine used here has a noise floor of roughly $\pm 30\%$ per instance even at per-instance minima over five repeats, so it cannot resolve a 6% effect, and no number of repeats fixes that. Counting integer programs and branch & bound nodes gives the same figure on every run and every machine, which is why Section 5.1 reports those and no seconds at all.

Two habits follow, and we adopted both only after failing at them. A measurement taken while anything else was running on the machine is discarded, not quoted — the first A/B for that section was contaminated exactly so and was re-run. And a gain predicted from the *mean* cost of a call is not a prediction: the profile that motivated the change showed the efficiency test at 18.4ms per call, the dearest operation in the search, but what the shortcut removes is the repair’s test at 7.68ms, the cheap kind. Reasoning from the wrong row of the right profile overstated the gain by half.

15 Summary of what was and was not gained

Table 17: Every hybridization attempted, with its verdict.

hybridization	verdict	effect
decision space $\rightarrow$ criterion space	win	$5.05\times$, up to $173\times$
exact–exact handover ($s = 1$)	tie	8.47 s vs 8.11 s
exact–exact handover ($s = 3$)	loss	13.14 s vs 8.47 s
exact–metaheuristic (Pareto archive)	win	$1.25\times$ of a $1.42\times$ ceiling
exact–Tchebychev seed	loss	$0.71\times$
Tchebychev as a generator	win	reaches unsupported points
incumbent seeding in decision space	loss	zero iterations saved
early exit on a hopeless bound	partial	25% fewer boxes, no time
early exit <i>on top of</i> the seed	loss	4% fewer boxes: a substitute
skipping the redundant test	win	6% fewer nodes, same search
warm-starting a box from its parent	win	11% fewer pivots, same search
cutting on a remembered efficient point	win	10% fewer pivots, 15 fewer boxes
zero-objective probe (a baseline check)	loss	4–13 $\times$ slower, same cuts
slices instead of an efficiency test	win	the whole of $E(P_D)$, under $2\times$

Of the fourteen, seven are wins, five are losses, one is a tie and one pays in sub-problems but not in time. They are reported at the same length as the wins because they carry the same information: a hybridization pays only where the exact method has a mechanism the first phase can feed, that mechanism is cheaper to identify than the hybrid is to build and measure, and two ideas feeding the same mechanism are one idea counted twice.

16 Prior work, and what remains to be checked

Criterion-space search for optimizing a function over the efficient set of a multi-objective integer program is an established family, and the surveys report it as currently more effective than decision-space search. Those methods are for *linear* objectives. The literature on a linear

fractional objective over an integer efficient set — including the branch-and-cut methods this note compares against, and the extensions to fractional criteria and to quadratic objectives — works in decision space. The augmented weighted Tchebychev program of Section 10 is standard and is used here exactly as the references use it; nothing about it is claimed as new. The combination treated in Section 3 appears not to have been addressed, for the reason given in Remark 1: Φ is a function of x and not of $Z(x)$, so the transfer is not immediate.

This is a statement about what we did not find, not a claim of novelty. A proper bibliographic review, including the sources behind paywalls, remains to be done before any such claim is made.

17 Reproducibility

The implementation is pure Python with no third-party dependency, computes in exact rational arithmetic throughout, and carries 80 tests. Several check properties rather than answers, since a method that silently lost efficient points would still agree with the optimum on favourable instances:

- Proposition 1 directly — for every centre and every feasible point, the point is either removed by the cut and in no child, or kept and in exactly one.
- Proposition 2 operationally — the metaheuristic is checked, over 14 instances and against an independent scan, never to hand over an inefficient point.
- Proposition 4 — every optimum of (10) is checked efficient against exhaustive enumeration, and the refusals ($\rho \leq 0$, a zero weight, fractional criteria) are checked to fire.
- Proposition 3 — every witness of a failed test on linear criteria is checked both efficient and dominating; the shortcut is checked to return exactly what the walk it replaces returns, so the centre never moves; and on fractional criteria the walk is checked to still be taken and still to end efficient.
- The early exit of Section 3.4 — the run must end with boxes still open and unsolved while staying proved optimal and agreeing with the independent scan, and a seeded run, where the exit fires sooner, must return the same value as an unseeded one.
- Proposition 5 — the enumerated front is checked against exhaustive enumeration, not a sample of it; each vector’s point is checked efficient and checked to attain that vector; the largest “Phi” on the front is checked to equal the optimum of (P_E) ; a budget-limited run is checked to refuse the completeness flag; and the whole of it is checked again on fractional criteria, where the centre comes from a dominance walk rather than from Proposition 3.
- The deliverable of Section 11 — every member is checked efficient and checked to carry the value Φ actually takes there, the proved optimum is checked to be present and ranked first, and each source is checked to work alone.
- The invariant of Section 5.2 — every point the search remembers is checked efficient, since the shortcut there cuts on one without re-testing it.
- Section 5.3 — a row added to a solved tableau is checked against rebuilding the model and solving it cold, on 120 random programs; and a child box’s warm root is checked to reach the same relaxation value as its cold one.
- The unsupported-reach claim — on the reference instance, the three unsupported efficient points are all reached by (10) while no positive weighted sum reaches any.

Every comparison in this note is reproduced by scripts in the repository, run on every commit by continuous integration, which assert that each seeded run returns the same proved optimum as the unseeded one.